



50+ лайфхаков, которые помогут ускорить работу с *агентами* для написания кода.

Постоянно пополняющаяся коллекция подсказок, рабочих процессов и ритуалов для разработчиков, использующих Claude Code , Codex  и Cursor .

🔍 Ищите лайфхаки, агентов, идеи...

Все агенты

 Клод Код

 Кодекс

 Курсор

Все

Контекст

Подсказки

Рабочий процесс

Автоматизация

Инструменты

КОНТЕКСТ

Закрепить файл CLAUDE.md в корне репозитория

Краткий файл CLAUDE.md автоматически загружается при каждом сеансе — используйте его для описания стиля репозитория, скриптов и нюансов, которые модель не может определить.

ХАКЕРСКИЙ ПРИЕМ #01

ЧИТАТЬ →

РАБОЧИЙ ПРОЦЕСС

Shift+Tab для перехода в режим планирования при внесении рискованных изменений

Режим планирования позволяет Claude свободно читать файлы, но блокирует их изменение — идеальный вариант для обсуждения подхода до внесения изменений в какой-либо файл.

ВЗЛОМ #02

ЧИТАТЬ →

ПОДСКАЗКА

Проекционные слэш-команды как многократно используемые подсказки

Поместите файл в формате Markdown в `.claude/commands/`, и он станет командой `/your-command` — идеально подходит для повторяющихся рабочих процессов.

ХАКЕРСКИЙ ПРИЕМ #03

ЧИТАТЬ →

РАБОЧИЙ ПРОЦЕСС

Распределите независимые поисковые запросы между субагентами

Запускайте несколько агентов Explore в одном сообщении — они работают параллельно и не влияют на основной контекст.

ХАКЕРСКИЙ ПРИЕМ #04

ЧИТАТЬ →

АВТОМАТИЗАЦИЯ

Блокировка остановки до прохождения тестов с помощью хука

Хук Stop может не завершать поворот, если тесты не пройдены, и возвращать модель в исходное положение до тех пор, пока работа не будет считаться выполненной.

ХАКЕРСКИЙ ПРИЕМ #05

ЧИТАТЬ →

ИНСТРУМЕНТЫ

Добавьте сервер MCP одной командой

Подключите Notion, Linear, Postgres или любой другой сервер MCP с помощью команды ``claude mcp add`` — редактирование конфигурационного файла не требуется.

ХАКЕРСКИЙ ПРИЕМ №06

ЧИТАТЬ →

КОНТЕКСТ

Слой `.cursor/rules/*.mdc` по области видимости

Правила `.mdc` для каждой папки с использованием шаблонов лучше, чем один гигантский файл `.cursorrules` — модель видит только то, что имеет отношение к редактируемому файлу.

ХАКЕРСКИЙ ПРИЕМ #07

ЧИТАТЬ →

ПОДСКАЗКА

Прикрепите файлы к чату Cursor с помощью `@`, прежде чем задавать вопрос

`@`-укажите 2–3 показательных файла перед тем, как задать вопрос. Функция поиска в Cursor работает хорошо, но для рефакторинга лучше использовать явное указание.

ХАКЕРСКИЙ ПРИЕМ #08

ЧИТАТЬ →

РАБОЧИЙ ПРОЦЕСС

Codex: разделяйте задачи по естественным границам коммитов

Интерфейс командной строки Codex лучше всего работает, когда каждая задача завершается коммитируемым diff. Не просите его «собрать всю функцию» — разбивайте ее на коммиты.

ХАКЕРСКИЙ ПРИЕМ #09

ЧИТАТЬ →

РАБОЧИЙ ПРОЦЕСС

Запускайте агентов в параллельных рабочих деревьях, а не в ветках

git worktree предоставляет каждому агенту собственную рабочую копию — никаких танцев с бубном, никаких незафиксированных изменений.

ХАКЕРСКИЙ ПРИЕМ №10

ЧИТАТЬ →

ПОДСКАЗКА

Напишите спецификацию, а затем попросите агента ее реализовать

Спецификация из 30 строк всегда лучше, чем запрос из трех строк — агенты следуют письменным спецификациям лучше, чем неформальному общению.

ХАКЕРСКИЙ ПРИЕМ №11

ЧИТАТЬ →

ПОДСКАЗКА

Вставьте скриншоты пользовательского интерфейса, чтобы понять замысел дизайна

Отправьте скриншот неработающего интерфейса (или фрейм Figma) прямо в чат — агенты гораздо лучше справляются с визуальной работой с пользовательским интерфейсом.

ХАКЕРСКИЙ ПРИЕМ №12

ЧИТАТЬ →

РАБОЧИЙ ПРОЦЕСС

claude --resume, чтобы продолжить вчерашнюю тему

Сеансы сохраняются локально — `claude --resume` выводит список предыдущих диалогов, чтобы вы могли продолжить длительную задачу спустя несколько дней.

ХАКЕРСКИЙ ПРИЕМ №13

ЧИТАТЬ →

Запустите Claude Code в автономном режиме в CI для сортировки

``claude -p`` — неинтерактивная команда. Подключите ее к CI, чтобы автоматически пометать проблемы, черновики описаний PR или тесты, не прошедшие первый этап проверки.

ХАКЕРСКИЙ ПРИЕМ №14

ЧИТАТЬ →

Разрешить выполнение команд только для чтения, чтобы избавиться от надоедливых запросов

Добавьте в настройки ``Bash(ls:*)``, ``Bash(rg:*)``, ``Bash(git status:*)`` и т. д. — агент перестанет запрашивать разрешение на безопасную проверку.

ХАКЕРСКИЙ ПРИЕМ №15

ЧИТАТЬ →

Принудительно создайте список TODO перед выполнением нетривиальной работы

Откройте с пометкой «сначала запланируйте как список TODO, а затем выполните» — агент сам разделит задачу на подзадачи, и вы получите контрольную точку для перенаправления.

НАСК #16

READ →

Debug frontend bugs with the Chrome DevTools MCP

Google's Chrome DevTools MCP gives Cursor a live Chrome instance — it can click, read the console, and pull network requests instead of guessing from screenshots.

НАСК #17

READ →

AUTOMATION

Watch background tasks with the Monitor tool

The Monitor tool replaces the old `/loop` workaround — Claude streams stdout from a background process and only spends tokens when an error appears.

HACK #18

READ →

WORKFLOW

Spawn parallel subagents for code review

Reviewing your own code with the agent that wrote it just rubber-stamps decisions — explicitly delegate to parallel subagents, each with a different lens.

HACK #19

READ →

AUTOMATION

Get notified when Claude Code finishes a task

Drop `.wav` files into `~/claude/hooks/` and let Claude wire up SessionStart/Notification/Stop hooks so you don't hover over the terminal.

HACK #20

READ →

AUTOMATION

Block API keys from leaking via a UserPromptSubmit hook

A pre-submit hook can grep your prompt for secret patterns and refuse to send if it finds one — safer than relying on memory.

HACK #21

READ →

WORKFLOW

Rewind a Codex session by double-tapping Esc

Double-tap Esc with an empty composer to edit your previous message — keep tapping to walk further back, Enter to fork a new branch from there.

HACK #22

READ →

AUTOMATION

Kill permission spam with /fewer-permission-prompts

Boris Cherny's skill scans your session history for safe Bash and MCP commands that always trigger prompts, then suggests an allowlist.

HACK #23

READ →

TOOLS

Catch type errors before commit with LSP plugins

Language Server plugins give Claude live diagnostics after every save — type errors and unused imports get fixed before you even read the diff.

HACK #24

READ →

AUTOMATION

Monitor deploys without leaving Claude Code

`/loop 5m check if the deploy succeeded` runs a recurring prompt in the background while you stay focused on real work.

HACK #25

READ →

CONTEXT

Lock down Claude Code's unstable behavior

Disable 1M context, adaptive thinking, and auto-memory in settings — and pin the subagent model to Sonnet — for noticeably more reliable runs.

HACK #26

READ →

PROMPTING

Pull live shell output into slash commands with !

Prefix a shell command with ! inside a ``.claude/commands/*.md`` file and Claude runs it first, then injects the output as context.

HACK #27

READ →

CONTEXT

Auto-load git context at session start

A ``SessionStart`` hook can dump ``git status`` to a known file so Claude always knows your branch and uncommitted changes without you typing it.

HACK #28

READ →

PROMPTING

Crank up reasoning with /effort high

Claude has been under-allocating thinking on hard tasks — ``/effort high`` (or ``max`` on Opus) plus ``showThinkingSummaries`` brings the deep analysis back.

HACK #29

READ →

PROMPTING

Embed live shell output into Claude Code skills

The same ``!` backtick syntax that works in slash commands also works in ``SKILL.md`` — your skills become live, not static.

HACK #30

READ →

TOOLS

Turn Figma frames into code with Codex

Codex's built-in Figma + Playwright skills pull design context from MCP, generate matching code, and iterate in a real browser until pixels line up.

HACK #31

READ →

CONTEXT

Feed Claude Code's own docs into context

Anthropic publishes a docs map at ``claude_code_docs_map.md`` — paste the URL and Claude grounds answers in current docs instead of stale training data.

HACK #32

READ →

PROMPTING

Make Claude Code grill you before building

Matt Pocock's ``/grill-me`` skill walks down the design tree question by question, surfacing every hidden assumption before a single line is written.

HACK #33

READ →

AUTOMATION

Run @codex review on every pull request

Enable Codex in your repo settings, comment `@codex review`, and it leaves inline comments like a teammate. `@codex fix it` opens a patch PR.

HACK #34

READ →

AUTOMATION

Configure Cursor's YOLO mode with a scoped allowlist

Don't just toggle YOLO mode — write a scoped prompt that whitelists tests and builds, deny destructive commands. Auto-runs without prompts.

HACK #35

READ →

TOOLS

Skill-route blocked URLs through Gemini

Claude's WebFetch hits 403s on Reddit. A custom skill detects the failure, opens a tmux Gemini session, and pulls the page through it.

HACK #36

READ →

CONTEXT

Auto-generate a tight CLAUDE.md with /init

An experimental `CLAUDE\_CODE\_NEW\_INIT` flag rewrites `/init` to scan your stack, linters, and CI — producing a 60-line CLAUDE.md instead of 400.

HACK #37

READ →

AUTOMATION

Schedule jobs in the cloud with `/schedule`

`/schedule` registers a recurring task that runs in Anthropic's cloud — no more 'my laptop closed and the job died.'

HACK #38

READ →

WORKFLOW

Promote any session into a reusable slash command

After running a workflow once, ask Claude to 'save what we just did into a new skill' — it writes the markdown and you get a slash command for free.

HACK #39

READ →

WORKFLOW

Run skills in isolated context with ``context: fork``

Adding ``context: fork`` and ``agent: Explore`` to a skill's frontmatter runs it in a subagent — your main thread only sees the final summary.

HACK #40

READ →

WORKFLOW

Collapse the PR ritual into one `/pr` command

Diff → commit → push → open PR is the same six commands every time. A ``.cursor/commands/pr.md`` file makes it ``/pr``.

HACK #41

READ →

TOOLS

Audit your Claude Code permissions with cc-safe

After weeks of approvals, your settings file is a minefield. `cc-safe` flags commands that could delete files, run as admin, or escalate access.

HACK #42

READ →

TOOLS

Strip needless abstractions with code-simplifier

Claude often adds unrequested abstractions. The official `code-simplifier` plugin refactors recent changes against your CLAUDE.md conventions.

HACK #43

READ →

WORKFLOW

Ask side questions mid-task with /btw

Mid-refactor and need to check something? `/btw` answers in a side panel without interrupting the main task or polluting context.

HACK #44

READ →

TOOLS

Share Claude Code sessions as HTML replays

`claude-replay` turns your `~/claude/projects/\*.jsonl` transcripts into a self-contained HTML player with speed controls and chapter bookmarks.

HACK #45

READ →

WORKFLOW

Fork a Claude Code session to try a different approach

``fork`` branches from the current point so you can experiment without losing context — both threads run independently.

HACK #46

READ →

AUTOMATION

Run prompts on repeat with scheduled tasks

``/loop 15m scan my error logs and flag anything new`` runs in the background while your session stays focused. Bare prose works for one-offs.

HACK #47

READ →

TOOLS

Cut Claude Code's token usage by 70% with rtk

Most context is noise — passing tests, progress bars, verbose logs. ``rtk`` is a CLI proxy that compresses command output before it hits the window.

HACK #48

READ →

AUTOMATION

Auto-format Claude Code's output with a PostToolUse hook

A ``PostToolUse`` hook on Write/Edit fires your formatter every time Claude touches a file — no more manual prettier/eslint runs before push.

HACK #49

READ →

PROMPTING

Stop agents from faking progress – demand proof in every status update

Agents love to say 'on it!' or 'done!' when nothing has actually run. Bind every status update to a process ID, file path, URL, or command output.

HACK #50

READ →

Made for developers shipping with agents.

Submit a hack → thecode@joinsuperhuman.io